

Technical Architecture Report: Production-Grade AI Social Media Automation System for Legal Domain Publishing

Executive Conclusion & Root Cause Analysis

The deployment failure of n8n on a Render Free container stems from an architectural mismatch between monolithic automation engines and constrained serverless container environments. n8n operates as an always-on Node.js process bundling an interactive UI canvas, execution engine, workflow state parser, and a massive dependency graph. Render Free enforces a strict memory ceiling of 512 MB of RAM without swap space. During container boot, engine initialization, and schema migrations over the Supabase Session Pooler, V8 heap requirements rapidly surpass the available memory. Garbage collection becomes ineffective, triggering a fatal V8 out-of-memory error (FATAL ERROR: Ineffective mark-compacts near heap limit). Attempting to run monolithic workflow daemons on 512 MB free container tiers is fundamentally unviable.

For event-driven, low-frequency workloads—such as generating and publishing one legal social media post per week—running an always-on orchestrator daemon is redundant and inefficient. The optimal, most reliable, and simplest architecture for 2026 is an **Asynchronous Event-Driven Serverless Pipeline based on Native Supabase Infrastructure (Supabase Cron + Supabase Edge Functions + Supabase Storage + PostgreSQL)**, with **Cloudflare Workflows** serving as the premier step-based alternative runtime.

Core Architectural Recommendations

- **Winning Architecture (Best Overall & Best \$0 Option): Native Supabase Serverless Stack.** Utilizing Supabase Cron (pg_cron) to trigger a sequential Deno/TypeScript Edge Function for legal research, LLM processing, image generation, and storage, paired with a dedicated Telegram Webhook Edge Function to handle human approval asynchronously. This architecture costs **\$0.00 per month** indefinitely, leverages existing infrastructure, avoids continuous memory overhead, and operates well within Supabase's 500,000 monthly free edge function invocations.
- **Best Simple Option: Cloudflare Workflows + Supabase Database.** Utilizing Cloudflare Workflows' native `step.sleep()` or `step.waitForEvent()` primitives. Cloudflare Workflows natively handles state persistence, pauses execution for human Telegram approval without consuming compute time, and resumes execution seamlessly.

System Requirements & Operational Boundaries

The system automates a weekly legal social media publication pipeline tailored specifically for Indian law topics. The automation engine executes once per week and enforces

human-in-the-loop validation prior to broadcast.

Stage	Functional Requirement	Technical Constraint	Primary Integration Point
1. Trigger	Scheduled execution once per week (e.g., Mondays at 08:00 IST).	Zero-cost scheduling with failure retry support.	Supabase Cron (pg_cron) / Cloudflare Cron.
2. Research	Scrape & query verified Indian legal data.	Prioritize primary official legal sources over third-party blogs.	India Code REST API, eSCR Supreme Court Portal, Press Information Bureau (PIB).
3. AI Synthesis	Topic selection, factual verification, caption formatting, hashtag selection, image prompt generation.	Must return strict JSON schema; low latency; zero operational cost.	Gemini 2.0 Flash / OpenRouter Free Tier.
4. Asset Creation	Render 1 high-quality visual/infographic per week.	Text clarity, commercial safety, square/portrait aspect ratio (1:1 or 4:5).	Pollinations.ai / Hugging Face Inference (FLUX.1-schnell).
5. Telegram Gate	Deliver draft summary, source citations, visual asset preview, and inline approval buttons to Telegram.	Interactive inline keyboard UI ([ Approve] [ Reject]).	Telegram Bot API (sendPhoto, InlineKeyboardMarkup).
6. Approval Pause	Pause workflow execution indefinitely until a human clicks an action button.	Must not consume compute time or hold open HTTP connections during wait.	Serverless Webhook Decoupling / Cloudflare Workflow State Machine.
7. Publishing	Broadcast approved media and content to Instagram, Facebook Page, and X (Twitter).	Compliance with official Meta Graph API & X API v2 requirements.	Meta Graph API v19.0+, X API v2 Free Tier.
8. Audit Logging	Record research metadata, verification status, media URLs, approval timestamps, and platform response payloads.	Persistent storage, transactional integrity.	Supabase PostgreSQL Database.

Primary Legal Source Integration Strategy

To ensure strict accuracy for legal content, the research ingestion stage bypasses third-party news blogs and queries official legal repositories directly:

1. **India Code Portal (indiacode.nic.in REST API)**: Provides access to central acts, state acts, section-level text, and subordinate legislation (rules, regulations, gazette notifications). Direct JSON endpoints (GET /api/v1/search?q=, GET

/api/v1/{act}/section/{number}) allow the system to extract exact statutory provisions.

2. **eSCR & Supreme Court Judgment Portals (scr.sci.gov.in / judgments.ecourts.gov.in)**: Enables programmatic querying of official Supreme Court of India neutral citations and decisions to validate statutory interpretations.
3. **Government Press Releases (PIB India & Ministry Data)**: Ingests press releases from the Ministry of Law and Justice, Ministry of Corporate Affairs (MCA), and Ministry of Home Affairs to track recent statutory amendments.

Evaluation of Core System Components

AI Reasoning & LLM Provider Evaluation

The pipeline requires an LLM capable of analyzing complex legal text, validating factual citations, generating social media captions, formulating image prompts, and outputting deterministic JSON.

Provider / Model	Free Tier Quota	Rate Limits	Structured JSON Support	Latency & Reliability	Suitability Verdict
Google Gemini 2.0 Flash / 1.5 Flash	Generous Free Tier via Google AI Studio API key.	15 RPM / 1,000,000 TPM / 1,500 RPD.	Native response_mime_type: "application/json" with schema enforcement.	Sub-second response time; 1M token context window for analyzing full acts.	Optimal Choice : Highest free quota, native JSON schema support, perfect for long legal texts.
OpenRouter (Free Tier Models)	Access to open models (e.g., meta-llama/llama-3.3-70b-instruct:free, deepseek/deepseek-r1:free).	Varies by model (typically 20-200 requests/day).	Supported via prompt engineering and standard OpenAI-compatible response format.	High variability during peak load on free endpoints; model availability fluctuates.	Secondary Backup : Reliable fallback mechanism if primary API experiences outages.
NVIDIA NIM API Catalog	Free developer credits upon registration.	1,000 free API calls total (trial credit).	High (supports standard OpenAI JSON schema).	High speed, enterprise-grade hardware backends.	Unsuitable for Long-Term \$0 Goal : Reverts to paid usage once trial credits deplete.
OpenAI (GPT-4o / GPT-4o-mini)	No permanent free API tier (requires prepaid billing balance).	Paid tier based on credit balance.	Industry-standard JSON mode and Structured Outputs.	Exceptional quality and low latency.	Paid Option Only : Violates \$0 requirement despite low cost (~\$0.005/post).

Image Generation API Evaluation

The system requires rendering one visually clear legal infographic or post image per week.

Provider / API	Cost Model & Free Tier Availability	Visual Quality & Text Rendering	API Integration & Speed	Commercial Terms	Overall Suitability
Pollinations.ai	100% Free ; no API key required; unmetered REST usage.	FLUX.1 / SDXL backends; strong prompt adherence; handles simple text layouts.	REST GET/POST endpoints (e.g., https://image.pollinations.ai/prompt/{prompt}); immediate generation.	Open usage policies.	Winner for \$0 Architecture: Simple URL-based generation, zero cost.
Hugging Face Inference API (FLUX.1-schnell)	Free tier available via standard Hugging Face API token.	Excellent text rendering and graphic realism.	Standard REST API POST endpoint; requires handling cold-start queuing.	Permissive open-weights license (Apache 2.0).	Strong Secondary \$0 Option: High visual quality for legal infographics.
Gemini / Imagen 3 API	Free tier access via Google AI Studio API key.	High-resolution realistic rendering and graphics.	Standard Google GenAI SDK integration.	Subject to Google Developer Terms of Service.	High-Quality Alternative: Free tier access integrated directly with Gemini LLM calls.
OpenAI DALL-E 3 API	Paid only (\$0.040 per Standard 1024x1024 image).	Superior typography handling and visual synthesis.	Standard OpenAI API images/generations endpoint.	Commercial ownership rights granted to creator.	Best Paid Option: Exceptional quality if a ~\$0.16/month budget is allocated.

Social Media Platform API Integrations

Automated multi-platform publishing requires adherence to platform authentication standards and media container workflows.

1. Instagram Graph API

- **Account Requirement:** Requires an **Instagram Professional (Business or Creator) Account** connected to a verified Facebook Page. Personal Instagram accounts cannot post via API.
- **Authentication:** Long-Lived Page Access Token (valid for 60 days, refreshable via system user tokens or automated refresh scripts) exchanged from a short-lived User Token via a Meta Developer App.

- **Publishing Workflow:**
 1. *Container Creation:* Send POST `/ig-user-id/media` passing `image_url` (must point to a publicly accessible HTTPS image stored in Supabase Storage) and caption. The API returns a `container_id`.
 2. *Status Check:* Send GET `/container-id?fields=status_code` to verify processing completion (FINISHED).
 3. *Publication:* Send POST `/ig-user-id/media_publish` passing `creation_id={container_id}`.
- **Cost & Quotas:** 100% Free. Limit of 25 API-published posts per 24-hour rolling window per account.

2. Facebook Graph API

- **Account Requirement:** Facebook Page (Business/Public Page).
- **Authentication:** Long-Lived Page Access Token.
- **Publishing Workflow:** Direct single-step request: send POST `/page-id/photos` with url (Supabase Storage public URL) and caption.
- **Cost & Quotas:** 100% Free for Page publishing.

3. X (Twitter) API v2

- **Account Requirement:** Developer Account with a Project app configured under the Free API Tier.
- **Authentication:** OAuth 2.0 Bearer Token / User Context (API Key, API Secret, Access Token, Access Token Secret with Read and Write permissions).
- **Publishing Workflow:**
 1. *Media Upload:* Send POST `https://upload.twitter.com/1.1/media/upload.json` (multipart/form-data) to obtain a `media_id_string`.
 2. *Tweet Creation:* Send POST `https://api.twitter.com/2/tweets` with payload `{"text": caption, "media": {"media_ids": [media_id_string]}}`.
- **Cost & Quotas:** **Free Tier Supported.** The v2 Free Tier allows **1,500 Tweet posts per month** at the app level and 50 Tweets per 24 hours per user context, easily accommodating the target volume of ~4 Tweets per month.

Hosting & Orchestration Platform Evaluation

The primary architectural objective is replacing n8n with an framework that prevents memory crashes, eliminates hosting fees, minimizes maintenance, and natively handles asynchronous human approval.

Analysis of Hosting Alternatives

A. n8n (Render vs. Railway vs. Self-Hosted)

- **Render Free:** Unviable. Crashes continuously with V8 out-of-memory errors due to strict 512 MB RAM ceilings.
- **Railway:** Unviable for \$0. Railway deprecated its permanent free tier in August 2024. It

currently offers a one-time \$5 trial credit lasting 30 days, after which the ongoing \$0/month Free tier provides only \$1/month in resource credits with a 0.5 GB RAM cap. Running n8n exhausts the \$1 credit in days, resulting in container suspension.

- **n8n Cloud:** Paid product starting at ~\$20+/month, exceeding budget requirements.

B. Render Workflows

Render Workflows are designed for transient background tasks. They bill based on active CPU and RAM compute time (\$0.05/hour starter or \$0.20/active CPU hour). They are unsuited for event-driven serverless waiting mechanisms. Pausing a workflow to await human Telegram approval requires either running an active polling worker (incurring compute charges) or managing external webhooks manually.

C. Cloudflare Workers & Workflows

- **Cloudflare Workers Free Tier:** Includes 100,000 requests per day and 128 MB RAM. While individual HTTP Worker invocations have a 10 ms CPU limit on the free tier, wall-clock I/O time (waiting for external APIs like LLMs or Telegram) is **unlimited**.
- **Cloudflare Workflows Free Tier:** Explicitly included in the Workers Free plan. Includes 3,000 step executions per day free and 1 GB-month of persisted state storage free. Crucially, Cloudflare Workflows support `step.sleep()` for up to 365 days and `step.waitForEvent()` natively. A step can execute research, store state, send a Telegram message, and enter an idle sleep state consuming **\$0 compute resources** until woken up by a Telegram webhook event.

D. Supabase Edge Functions + Supabase Cron (Native Stack)

- **Resource Allowance:** Supabase Free Tier includes 500,000 Edge Function invocations per month, 500 MB PostgreSQL database storage, 1 GB file storage, and 5 GB outbound egress.
- **Execution Environment:** Deno/TypeScript serverless runtime. Edge Functions feature a 150-second wall-clock timeout on the free tier and 256 MB of RAM.
- **Orchestration Architecture:** Because serverless functions are ephemeral, long-running workflows are decoupled into discrete functions: * *Function 1 (Generator):* Triggered weekly by Supabase Cron (`pg_cron`). Executes research, calls the LLM, generates the image, saves the draft to Supabase DB, posts to Telegram with callback buttons, and exits within ~10–15 seconds.
 - *Function 2 (Telegram Webhook Listener):* Listens for incoming POST requests from Telegram when an admin clicks Approve or Reject. Updates DB status and immediately invokes Function 3.
 - *Function 3 (Publisher):* Reads approved post payload from DB, publishes to Instagram, Facebook, and X, logs final status, and exits.
- **Verdict:** Highly efficient, zero-cost architecture that reuses pre-existing database infrastructure.

E. GitHub Actions

- **Resource Allowance:** 2,000 free build minutes per month for public/private repositories

on standard Linux runners (2 vCPU, 7 GB RAM).

- **Mechanics:** Scheduled workflows via cron syntax can trigger research and post generation. However, waiting for human approval inside a running GitHub Action workflow is an anti-pattern: it consumes runner minutes while idling and risks timing out (maximum workflow run time is 6 hours). Using GitHub Actions purely as a cron trigger to hit an external API endpoint is functional, but redundant if using Supabase Cron.

F. Vercel / Next.js Serverless Functions

- **Resource Allowance:** Vercel Free Tier includes 100 GB bandwidth, Vercel Cron (1 free cron job per project), and Serverless Functions with a 10-second default timeout (configurable up to 60s max on Hobby plan).
- **Limits:** The 10–60 second maximum execution limit can cause failures if multi-step AI synthesis, legal scraping, image generation, and asset uploads experience network latency. Vercel is optimized for frontend web applications rather than backend state-machine orchestration.

Comprehensive Architecture Comparison Table

Candidate Option	Free Tier?	Allocated RAM	Allocated CPU	Scheduling Mechanism	Durable Workflow?	Human Approval Mechanism	Webhooks Supported?	AI APIs Supported?	Image APIs Supported?	DB Integration	Operations & Maintenance	System Reliability	Architecture Verdict
n8n + Render Free	No	512 MB	0.1 vCPU	Internal Cron	Yes (Postgres)	Native Wait Node	Excellent	External	External	Supabase Pooler	High (Fatal Crashes)	Unusable (0/5)	REJECTED: Fatal V8 heap memory crash (out of memory).
n8n + Railway	No (\$5 trial only)	512 MB	1.0 vCPU	Internal Cron	Yes (Postgres)	Native Wait Node	Excellent	External	External	Supabase Pooler	Medium	High (until credit depleted)	REJECTED: Deprecated permanent free tier; credit expires after

Candidate Option	Free Tier?	Allocated RAM	Allocated CPU	Scheduling Mechanism	Durable Workflow?	Human Approval Mechanism	Webhooks Supported?	AI APIs Supported?	Image APIs Supported?	DB Integration	Operations & Maintenance	System Reliability	Architecture Verdict
													days.
Render Workflows	No (\$0.05+/hr)	512MB-16GB	0.5-4 vCPU	Render Scheduler	No (Requires DB)	Manual Polling / Custom	Good	External	External	Native / External	Low	High	REJECTED: Paid compute usage model; unsuited for serverless wait states.
Supabase Edge Stack	Yes (Free Forever)	256 MB	Shared	Supabase Cron (pg_cron)	Yes (Supabase DB)	Decoupled Async Webhooks	Excellent	External	External	Native PostgreSQL	Zero Maintenance	Maximum (5/5)	WINNER (Best Overall & \$0): Native, simple, zero cost, decoupled.
Cloudflare Workflows	Yes (Free Tier)	128 MB	10ms CPU / Wall Unlimited	Cloudflare Cron	Native State Persistence	Native step.waitForEvent()	Native	External	External	Supabase Client	Very Low	Maximum (5/5)	WINNER (Best Simple Option): Native pause/resume state machine.

Candidate Option	Free Tier?	Allocated RAM	Allocated CPU	Scheduling Mechanism	Durable Workflow?	Human Approval Mechanism	Webhooks Supported?	AI APIs Supported?	Image APIs Supported?	DB Integration	Operations & Maintenance	System Reliability	Architecture Verdict
GitHub Actions	Yes (2,000 mins/mo)	7 GB	2 vCPU	GitHub Scheduled Cron	No (Artifacts only)	Issues / PR Gate (Awkward)	Limited	External	External	Supabase SDK	Low	High	REJECTED: Inefficient waiting mechanics; 6-hour execution limits.
Vercel / Next.js	Yes (Hobby)	1024 MB	Shared	Vercel Cron (1 job)	No (Requires DB)	Async API Route Webhooks	Good	External	External	Supabase SDK	Low	Medium (10s Timeout)	REJECTED: Strict execution timeouts risk failure during AI image generation.

Detailed Top 3 System Architecture Designs

Architecture 1: Native Supabase Serverless Architecture (Best Overall & Best \$0 Option)

This architecture contains zero external workflow orchestrators, running fully within the user's pre-existing Supabase project.

System Execution Flow

1. **Trigger Phase:** Supabase Cron (pg_cron) executes once per week at a specified time (e.g., Monday 08:00 UTC). It invokes the research-generator Edge Function via a secure HTTP POST request using pg_net.
2. **Research & Ingestion Phase:** The research-generator function queries the India Code API (indiacode.nic.in) for statutory updates and the eSCR Supreme Court portal for legal judgments.

3. **AI Reasoning Phase:** Extracted raw texts are passed to the Gemini 2.0 Flash API with a strict JSON response schema enforcing topic extraction, fact verification, caption formatting, hashtags, and an image prompt.
4. **Visual Rendering Phase:** The function calls Pollinations.ai using the generated prompt, retrieves the binary image buffer, and uploads it directly to a public Supabase Storage bucket (media-assets).
5. **Database Registration:** The function inserts a record into the draft_posts PostgreSQL table with status pending_approval and stores the public image URL.
6. **Telegram Notification:** The function sends an HTTP POST request to the Telegram Bot API (sendPhoto) containing the draft image, legal summary, source links, and an inline approval keyboard ([ Approve] and [ Reject]). The function then exits statelessly.
7. **Human Approval Wait State:** The system consumes zero CPU resources while waiting. The draft state rests securely in the Supabase PostgreSQL database.
8. **Asynchronous Resume & Publishing:** When the user taps Approve in Telegram, Telegram posts a callback webhook payload to the telegram-webhook Supabase Edge Function. The function authenticates the Telegram cryptographic secret header, updates the database status to approved, and triggers the social-publisher Edge Function.
9. **Multi-Platform Dispatch:** The social-publisher Edge Function reads the approved post payload, calls the Instagram Graph API, Facebook Graph API, and X API v2 in parallel, records platform response IDs in publication_logs, and updates the Telegram message caption to confirm publication.

Architecture 2: Cloudflare Workflows + Supabase Backend (Best Simple Option)

This architecture uses Cloudflare Workflows as a stateful serverless orchestrator with built-in sleep and event-waiting capabilities.

System Execution Flow

1. **Scheduled Trigger:** A Cloudflare Cron Trigger fires weekly, initializing an instance of LegalPostWorkflow.
2. **Step 1 (research-and-ai):** The workflow executes legal scraping against India Code and eSCR, calls Gemini 2.0 Flash for structured content generation, requests image rendering from Pollinations.ai, and uploads the resulting image to Supabase Storage.
3. **Step 2 (notify-telegram):** The workflow dispatches an interactive Telegram message containing the draft details, source URLs, image preview, and callback buttons.
4. **Step 3 (waitForEvent):** The workflow executes await step.waitForEvent('telegram-approval-event', { t: '7 days' }). Cloudflare automatically serializes the execution state into persistent workflow storage (1 GB included free) and pauses compute execution. CPU consumption drops to **0 ms**.
5. **Step 4 (Webhook Resumption):** When the human clicks Approve in Telegram, Telegram hits a Cloudflare Worker webhook endpoint. The Worker validates the payload and calls instance.emitEvent('telegram-approval-event', payload) using the Cloudflare Workflow binding. The workflow resumes execution immediately at Step 5.

6. **Step 5 (publish-socials):** The workflow executes API calls to Instagram, Facebook, and X, writes the final log entry to the Supabase PostgreSQL database, and completes.

Architecture 3: GitHub Actions Orchestrator + Supabase Backend

This architecture uses GitHub Actions as an external runner triggered by a scheduled cron event.

System Execution Flow

1. **Cron Schedule:** A GitHub Actions workflow (.github/workflows/weekly-legal-post.yml) is triggered by a schedule event (cron: '0 2 * * 1').
2. **Generation Runner Execution:** A short-lived GitHub runner (Ubuntu latest) checks out the repository and executes a TypeScript script (ts-node generate-draft.ts).
3. **Pipeline Ingestion & AI:** The script fetches legal source data from official portals, invokes Gemini 2.0 Flash for structured content, renders an image via Pollinations.ai, uploads the asset to Supabase Storage, and writes the draft record to the Supabase PostgreSQL database.
4. **Telegram Dispatch & Runner Exit:** The script calls the Telegram Bot API to send the approval card and exits within ~60 seconds, using minimal free GitHub build minutes.
5. **Decoupled Webhook Publishing:** The waiting state and approval resumption are offloaded to a Supabase Edge Function (telegram-webhook). When the user approves the post in Telegram, the Supabase Edge Function processes the webhook, updates the database, and executes social publishing directly to Instagram, Facebook, and X.

Recommended Architecture & Decision Criteria

1. Best Overall Architecture: Native Supabase Serverless Architecture

Justification: Native Supabase (Supabase Cron + Edge Functions + Supabase Storage + Database) is the best overall solution. The user already operates an active Supabase project and database. Introducing external orchestrators like n8n, Cloudflare, or GitHub Actions adds unnecessary cross-cloud complexity, secret duplication, and network latency. Supabase Edge Functions access the database directly using the pre-configured service-role context, execute TypeScript natively via Deno, and consume less than 0.01% of the 500,000 monthly free execution quota for a 4-run-per-month workload.

2. Best \$0 Option: Native Supabase Serverless Architecture

Justification: Every component in Architecture 1 is permanently free:

- **Host Compute:** Supabase Edge Functions (500,000 free invocations/month).
- **Database & Storage:** Supabase Postgres (500 MB free) + Supabase Storage (1 GB free).
- **Cron Execution:** Supabase pg_cron (Included free in Postgres extensions).
- **Reasoning AI:** Google Gemini 2.0 Flash (Free Tier via AI Studio).

- **Image Asset Rendering:** Pollinations.ai (100% free, unmetered).
- **Human Interface:** Telegram Bot API (100% free, unmetered).
- **Social APIs:** Meta Graph API & X API v2 Free Tier (100% free for low-volume posting).
- **Total Estimated Monthly Infrastructure Spend: \$0.00 / month.**

3. Best Simple Option: Cloudflare Workflows + Supabase Backend

Justification: For developers who prefer an imperative step-based coding abstraction where sequential code -> wait -> publish execution is declared in a single file without decoupling into separate webhook listening functions, Cloudflare Workflows is the simplest option. Its native handling of durable state persistence and `step.waitForEvent()` eliminates manual state transition management in the database.

Detailed End-to-End Workflow Execution Sequence

The complete weekly automation lifecycle operates through the following step-by-step sequence:

Step Number	Component	Executing Entity	Detailed Operation	State Transition
1.0	Schedule Trigger	Supabase pg_cron	Fires cron event at 0 2 * * 1 (Mondays 07:30 IST). Calls pg_net to issue HTTP POST to Edge Function.	None
2.0	Legal Fetch	research-generator	Queries India Code API (/api/v1/search) and eSCR Supreme Court portal for legal topic data.	status = 'researching'
3.0	AI Synthesis	research-generator	Passes raw text to Gemini 2.0 Flash API with strict JSON schema enforcing factual extraction and caption formatting.	status = 'synthesizing'
4.0	Image Render	research-generator	Sends prompt to Pollinations.ai, downloads binary buffer, and uploads image to Supabase Storage (media-assets/).	Image stored in Storage bucket
5.0	Draft Storage	research-generator	Writes record into draft_posts	status = 'pending_approval'

Step Number	Component	Executing Entity	Detailed Operation	State Transition
			2](start_span)[span_132](end_span)ts PostgreSQL table with generated metadata, source URLs, and image URL.	
6.0	Admin Alert	research-generator	Calls Telegram API sendPhoto with image, draft text, source citations, and inline callback buttons ([ Approve] / [ Reject]).	Draft message sent to Telegram
7.0	Idle Wait State	System Backend	Stateless pause. Serverless execution completes. Edge function exits. Zero compute resources consumed.	DB holds pending[span_227](start_span)[span_227](end_span)[span_230](start_span)[span_230](end_span)ng_approval
8.0	Human Action	Telegram Client	Admin reviews draft, verifies legal source URLs, and taps [ Approve] inside Telegram app.	User triggers callback query
9.0	Webhook Process	telegram-webhook	Telegram hits webhook endpoint. Function validates secret header, extracts draft_id, and updates database.	status = 'approved'
10.0	Multi-Publish	social-publisher[span_43](start_span)[span_43](end_span)[span_81](start_span)[span_81](end_span)]	Reads approved draft from DB. Issues API calls to Instagram Graph API, Facebook Graph API, and X API v2 in parallel.	status = 'publishing'
11.0	Audit Logging	social[span_44](start	Records social	status =

Step Number	Component	Executing Entity	Detailed Operation	State Transition
		rt_span)[span_44](end_span)[span_82](start_span)[span_82](end_span)-publisher	platform post IDs and raw JSON responses in publication_logs table.	'published'
12.0	Telegram Edit	social-publisher	Edits original Telegram message caption to remove buttons and display: "✔ Published Successfully!".	Complete

Telegram Approval Interface & Webhook Design

Webhook Setup & Security Registration

To register the webhook endpoint with Telegram, issue an HTTP POST request to the Telegram Bot API:

https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/setWebhook

Payload:

```
{
  "url":
  "https://<supabase-project-ref>.supabase.co/functions/v1/telegram-webhook",
  "secret_token": "CRITICAL_CRYPTOGRAPHIC_SECRET_STRING_32_CHARS"
}
```

Interactive Telegram Card Payload (Dispatched by research-generator)

```
{
  "chat_id": "<ADMIN_TELEGRAM_CHAT_ID>",
  "photo":
  "https://<supabase-ref>.supabase.co/storage/v1/object/public/media-assets/drafts/2026-w10.png",
  "caption": "🏛️ *NEW LEGAL CONTENT DRAFT FOR APPROVAL*\n\n🔴 *Topic:* The Bharatiya Sakshya Adhiniyam, 2023 (Section 61 - Electronic Evidence)\n\n⚖️ *Key Takeaway:* Primary evidence explicitly [span_45] (start_span) [span_45] (end_span) [span_83] (start_span) [span_83] (end_span)tly includes semiconductor memory and smartphone storage.\n\n🔗 *Primary Sources:* \n- indiacode.nic.in/handle/123456789/20062\n- scr.sci.gov.in/pdfsearch\n\n📝 *Caption Draft:* \nDid you know your
```

phone storage is now legally classified as primary evidence in court?
Here is what Section 61 of the BSA 2023 changes for digital forensics.\n\n#IndianLaw #BSA2023 #LegalTech #SupremeCourt",

```
"parse_mode": "Markdown",
"reply_markup": {
  "inline_keyboard": [
    [
      { "text": "✅ Approve & Publish", "callback_data":
"action=approve&id=c9b1d8f2-8e2b-4b1a-9f2d-1a2b3c4d5e6f" },
      { "text": "❌ Reject Draft", "callback_data":
"action=reject&id=c9b1d8f2-8e2b-4b1a-9f2d-1a2b3c4d5e6f" }
    ]
  ]
}
```

Database Design & SQL Schema Specification

The following relational database schema is deployed to Supabase PostgreSQL to support state tracking, legal verification tracing, and publishing audit logs:

```
-- Enable UUID generation extension
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

-- Define Custom Enum Types for System Status Tracking
CREATE TYPE post_status AS ENUM (
  'draft_generated',
  'pending_approval',
  'approved',
  'rejected',
  'publishing',
  'published',
  'failed'
);

CREATE TYPE platform_type AS ENUM ('instagram', 'facebook',
'twitter');

-- Core Table: Legal Content Research, Drafts, and Visual Assets
CREATE TABLE draft_posts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  status post_status DEFAULT 'draft_generated',

  -- Legal Research & Verification Metadata
  topic_title TEXT NOT NULL,
```

```

    act_name TEXT,
    section_number TEXT,
    primary_source_urls TEXT[] NOT NULL,
    legal_rationale TEXT NOT NULL,
    fact_verification_summary TEXT NOT NULL,

    -- Content Output Payload
    caption_text TEXT NOT NULL,
    hashtags TEXT[] NOT NULL,
    image_prompt TEXT NOT NULL,
    image_storage_path TEXT,
    image_public_url TEXT,

    -- Telegram Interaction Metrics
    telegram_message_id BIGINT,
    approved_at TIMESTAMP WITH TIME ZONE,
    rejected_at TIMESTAMP WITH TIME ZONE,
    rejection_reason TEXT
);

-- Audit Table: Multi-Platform Publishing Logs
CREATE TABLE publication_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    draft_id UUID REFERENCES draft_posts(id) ON DELETE CASCADE,
    platform platform_type NOT NULL,
    published_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    platform_post_id TEXT,
    api_response_payload JSONB,
    status TEXT NOT NULL,
    error_message TEXT
);

-- Performance Indexes for Webhook Lookup Queries
CREATE INDEX idx_draft_posts_status ON draft_posts(status);
CREATE INDEX idx_draft_posts_telegram_msg ON
draft_posts(telegram_message_id);

```

AI Reasoning & Visual Asset Design

Legal Research Verification Prompt (Gemini 2.0 Flash)

```

// Prompt Construction for Gemini 2.0 Flash Legal Verification Engine
const legalResearchPrompt = `
You are an expert Indian Legal Analyst and Social Media Content
Strategist.
Analyze the following raw legal text retrieved from primary Indian

```

legal sources (India Code API and eSCR Supreme Court Judgments):

RAW STATUTORY DATA:

`${indiaCodeRawText}`

RAW JUDGMENT DATA:

`${escrRawText}`

INSTRUCTIONS:

1. Select the single most relevant and impactful legal topic for public legal awareness.
2. Cross-verify all legal claims strictly against the statutory sections and judgment text provided.
3. Reject any unverified speculation, editorial opinion, or news blog claims.
4. Output a strict JSON object following this exact schema:

```
{
  "topic_title": "Short descriptive topic title",
  "act_name": "Official Title of the Legislation",
  "section_number": "Section X / Article Y",
  "primary_source_urls": ["https://indiacode.nic.in/...",
"https://scr.sci.gov.in/..."],
  "fact_verification_summary": "Summary of verified statutory
evidence",
  "caption_text": "Engaging, professional social media caption
structured with clear paragraphs",
  "hashtags": ["#IndianLaw", "#LegalAwareness", "#SupremeCourt"],
  "image_prompt": "Clean graphic design prompt for a legal visual,
minimalist style, dark theme, crisp vector illustration, high
contrast, text banner reading the statutory topic"
}
```

`;

Social Media Publishing API Integration Specifications

Multi-Platform Publishing Logic (social-publisher Edge Function)

```
// Social Media Publisher Execution Engine
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

async function publishToInstagram(imageUrl: string, caption: string):
Promise<{ id: string }> {
  const igUserId = Deno.env.get("INSTAGRAM_USER_ID")!;
  const accessToken = Deno.env.get("META_LONG_LIVED_PAGE_TOKEN")!;
```

```

    // Step 1: Create Container
    const containerRes = await
    fetch(`https://graph.facebook.com/v19.0/${igUserId}/media`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            image_url: imageUrl,
            caption: caption,
            access_token: accessToken,
        }),
    });
    const containerData = await containerRes.json();
    if (!containerRes.ok) throw new Error(`Instagram Container Error:
    ${JSON.stringify(containerData)}`);

    // Step 2: Publish Container
    const publishRes = await
    fetch(`https://graph.facebook.com/v19.0/${igUserId}/media_publish`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            creation_id: containerData.id,
            access_token: accessToken,
        }),
    });
    const publishData = await publishRes.json();
    if (!publishRes.ok) throw new Error(`Instagram Publish Error:
    ${JSON.stringify(publishData)}`);

    return { id: publishData.id };
}

async function publishToFacebook(imageUrl: string, caption: string):
Promise<{ id: string }> {
    const pageId = Deno.env.get("FACEBOOK_PAGE_ID")!;
    const accessToken = Deno.env.get("META_LONG_LIVED_PAGE_TOKEN")!;

    const res = await
    fetch(`https://graph.facebook.com/v19.0/${pageId}/photos`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            url: imageUrl,
            message: caption,
            access_token: accessToken,
        }),
    });
    const data = await res.json();

```

```

    if (!res.ok) throw new Error(`Facebook Publish Error:
    ${JSON.stringify(data)}`);
    return { id: data.id };
}

```

Security, Secrets Management, & Resiliency

Secret Management Protocol

- **Zero Secrets in Code:** All API credentials (Google AI Studio Key, Telegram Bot Token, Meta Access Tokens, X API Tokens, Supabase Service Role Keys) are stored in **Supabase Secrets** managed via the Supabase CLI (supabase secrets set KEY=VALUE).
- **Telegram Webhook Authorization:** The Telegram Webhook listener validates the X-Telegram-Bot-API-Secret-Token header on every incoming POST request, preventing unauthorized entities from forging approval events.

Idempotency & Retry Architecture

- **State Safeguards:** The database maintains state transitions (pending_approval -> approved -> publishing -> published). If a Telegram callback button is accidentally tapped multiple times, the database transaction enforces state validation (WHERE status = 'pending_approval'), preventing duplicate social posts.
- **Transient Error Handling:** Outbound HTTP requests to external APIs (LLM, Image Generation, Social APIs) use exponential backoff retries (3 retries with 2s, 4s, and 8s delays) to mitigate transient network drops.

Estimated Monthly Operating Cost Breakdown

Component	Platform / Service	Free Tier Allowance	Project Monthly Usage	Monthly Cost
Compute Engine	Supabase Edge Functions	500,000 requests / month	~12 function calls	\$0.00
Database & Cron	Supabase PostgreSQL + pg_cron	500 MB DB storage	~5 MB storage	\$0.00
Asset Storage	Supabase File Storage	1 GB storage	~8 MB storage	\$0.00
Reasoning Engine	Google Gemini 2.0 Flash API	1,500 requests / day free	~4 requests / month	\$0.00
Image Render API	Pollinations.ai / Hugging Face	100% Free / Unmetered	~4 images / month	\$0.00
Telegram Gateway	Telegram Bot API	100% Free / Unmetered	~10 messages / month	\$0.00
Social Distribution	Instagram, Facebook & X	Free Tier Allowances	~4 posts per platform	\$0.00

Component	Platform / Service	Free Tier Allowance	Project Monthly Usage	Monthly Cost
	APIs			
TOTAL MONTHLY COST				\$0.00 / month

Step-by-Step Implementation Roadmap

1. Step 1: Database Initialization

- Access the Supabase Dashboard SQL Editor.
- Execute the SQL DDL schema provided in this report to establish tables (draft_posts, publication_logs) and custom ENUM types.
- Enable the pg_cron and pg_net extensions under Project Settings -> Extensions.

2. Step 2: Storage Bucket Configuration

- Create a public Supabase Storage Bucket named media-as[span_51](start_span)[span_51](end_span)[span_89](start_span)[span_89](end_span)sets.
- Configure bucket policies to permit public read access so social media Graph APIs can download published post images.

3. Step 3: Telegram Bot Setup

- Message @BotFather on Telegram to generate a bot and receive the TELEGRAM_BOT_TOKEN.
- Create an administrative Telegram Group/Channel, add the bot, and obtain the TELEGRAM_CHAT_ID.

4. Step 4: Social Media Developer Portal Setup

- Create a Meta Developer App, link an Instagram Professional Account and Facebook Page, and generate a Long-Lived Page Access Token with instagram_content_publish and pages_manage_posts permissions.
- Create an X Developer account under the Free Tier, generating OAuth 2.0 credentials with Write access.

5. Step 5: Edge Function Deployment

- Initialize Supabase CLI locally: supabase init.
- Create functions: supabase functions new research-generator, supabase functions new telegram-webhook, and supabase functions new social-publisher.
- Populate function code with Deno TypeScript implementation logic.
- Upload environment secrets: supabase secrets[span_55](start_span)[span_55](end_span)[span_93](start_span)[span_93](end_span) set GEMINI_API_KEY=... TELEGRAM_BOT_TOKEN=... META_LONG_LIVED_PAGE_TOKEN=....
- Deploy functions: supabase functions deploy.

6. Step 6: Register Webhook & Schedule Cron

- Issue a POST request to Telegram's setWebhook endpoint referencing the telegram-webhook function URL.
- Schedule the weekly cron job in Supabase SQL Editor:

```
SELECT cron.schedule(
  'weekly-legal-post-job',
  '0 2 * * 1',
  $$ SELECT net.http_post(
```

```
url:='https://<project-ref>.supabase.co/functions/v1/research-generator',
  headers:='{ "Content-Type": "application/json",
"Authorization": "Bearer <SERVICE_ROLE_KEY>" }'::jsonb
) $$
);
```

Technologies to Exclude and Anti-Patterns

- **Do NOT attempt to run n8n on Render Free or Railway Free:** n8n's monolithic Node.js runtime exceeds Render Free's 512 MB memory limit during boot and schema migration, triggering V8 heap crashes (FATAL ERROR: Ineffective mark-compacts near heap limit). Railway deprecated its permanent free tier, and its \$1/month post-trial credit expires within days.
- **Do NOT use personal Instagram accounts:** The Instagram Graph API explicitly rejects personal accounts. Publishing requires an Instagram Professional (Business or Creator) account linked to a Facebook Page.
- **Do NOT hold serverless functions open while waiting for human approval:** Keeping an HTTP function sleeping or polling while waiting for Telegram interaction burns execution time limits and incurs unnecessary compute charges. Always use stateless, decoupled webhook callbacks.
- **Do NOT rely on unverified third-party legal blogs:** Parsing secondary editorial blogs introduces significant factual hallucination risks. The ingestion pipeline must pull primary statutory text directly from official legal infrastructure like the India Code REST API and Supreme Court eSCR resources.

Direct Answer to Core Architectural Question

"What is the most modern, reliable, simplest and preferably completely free way to build this weekly AI research → content → image → Telegram approval → Instagram/Facebook/X publishing system without running into the RAM/resource problems I encountered with n8n on Render Free?"

The most modern, reliable, simplest, and **100% free** way to build this system in 2026 is to **abandon monolithic orchestrators like n8n entirely and use the Native Supabase Serverless Stack (Supabase Cron + Supabase Edge Functions + Supabase Storage + PostgreSQL).**

Why This Solution Directly Solves the Problem

1. **Zero RAM / Memory Crashes:** Instead of hosting a heavy 512 MB Node.js n8n container on Render that crashes V8 heap limits during migrations, Supabase Edge Functions run transient Deno isolates that consume minimal RAM (well under their 256 MB ceiling) and execute in seconds before terminating.
2. **Built-in Decoupled Human Approval:** The workflow does not hold open HTTP connections or run continuously while waiting for Telegram approval. Function 1

generates the draft, alerts Telegram, updates the Supabase DB, and shuts down. When the user taps Approve in Telegram, Telegram calls Function 2 via webhook, which updates the database and triggers Function 3 to publish to Instagram, Facebook, and X.

3. **100% Free Forever (\$0.00/month):** The workload (~4 executions per month) uses less than 0.01% of Supabase's free tier (500,000 Edge Function calls, 500 MB Postgres DB, 1 GB File Storage), Google Gemini's free API tier (1,500 requests/day), Pollinations.ai's unmetered image generation, Telegram's free Bot API, and Meta/X free API tiers.
4. **Reuses Existing Setup:** It leverages the user's pre-existing Supabase project without introducing third-party container hosts, paid infrastructure, or external orchestration tools.

Works cited

1. Render Pricing Guide: Free Tier, Compute and Database Costs, <https://kuberns.com/blogs/render-pricing/>
2. Pricing - Render, <https://render.com/pricing>
3. Railway Free Tier 2026: Plans, Credits and Limits - Kuberns, <https://kuberns.com/blogs/railway-free-tier/>
4. Limits · Cloudflare Workflows docs, <https://developers.cloudflare.com/workflows/reference/limits/>
5. Pricing · Cloudflare Workflows docs, <https://developers.cloudflare.com/workflows/reference/pricing/>
6. Supabase Pricing 2026: Free Tier Limits & Real Costs, <https://designrevision.com/blog/supabase-pricing>
7. Supabase Free Tier Limits (2026): Database, Storage & Projects, <https://www.itpathsolutions.com/supabase-free-tier-limits>
8. India Code API – Indian Legislation Data - Parse.bot, <https://parse.bot/marketplace/bdf0b8e9-8f2b-4ac4-b4a6-652f3b8323c4/indiacode-nic-in-api>
9. How to Find Any Indian Act or Section: IndiaCode Search, JSON API, <https://blogs.ecourtsindia.com/2026/08/30/indiacode-search-json-api-guide/>
10. Search Results - India Code, https://www.indiacode.nic.in/handle/123456789/1362/simple-search?query=&sort_by=score&order=asc&rpp=85&filter_field_1=department&filter_type_1>equals&filter_value_1=+States&filter_field_2=actyear&filter_type_2>equals&filter_value_2=%5B1836+TO+1899%5D&filter_field_3=ministry&filter_type_3>equals&filter_value_3=Home+Affairs&filter_field_4=title&filter_type_4>equals&filter_value_4=Bengal+Ghatwali+Land+Act%2C+1859&etal=0&filtername=department&filterquery=States&filtertype>equals
11. About Us - eSCR, https://judgments.ecourts.gov.in/pdfsearch/?p=pdf_search/footerlinks/ab&app_token=031b0edf59d50853ed5e9604b1030d470cd649d199bf9da08f6e6099fbf08385
12. Home | Judgements and Orders, Supreme Court and High courts of, <https://scr.sci.gov.in/>
13. Railway Free Tier 2026: \$5 Trial Credit Guide - Antilak, <https://antilak.com/blog/railway-free-tier-2026-5-dollar-trial>
14. Pricing - Railway, <https://railway.com/pricing>
15. Railway Pricing 2026: Plans, Costs and Alternatives - Temps, <https://temps.sh/compare/vs-railway>
16. Cloudflare Free Tier Limits Checklist: Are CDN, DNS, WAF, and, <https://eastonddev.com/blog/en/posts/dev/20260526-cloudflare-free-limits/>
17. Limits · Cloudflare Workers docs, <https://developers.cloudflare.com/workers/platform/limits/>
18. Cloudflare Pages Pricing 2026: Free Tier Limits, Workers Costs, <https://nayankyada.com/blog/cloudflare-pages-pricing-2026-free-tier-limits-workers-costs-when-to-upgrade>
19. Workflows pricing adds per-step billing. Step and storage billing to, <https://developers.cloudflare.com/changelog/post/2026-07-07-workflows-billing-updates/>
20. Edge Function 'wall clock time limit reached' - Supabase, <https://supabase.com/docs/guides/troubleshooting/edge-function-wall-clock-time-limit-reached-Nk38bW>
21. Limits | Supabase Docs, <https://supabase.com/docs/guides/functions/limits>
- 22.

Play | pollinations.ai, <https://pollinations.ai/play>